BEAM: Building Energy Assessment Model

[*]Submitted as the final project for the ARTINT course.

Nilabh Pandey
School of Computer Science and Information Technology
Lucerne University of Applied Sciences and Arts
Lucerne, Switzerland
nilabh.pandey@stud.hslu.ch

***Abstract*—Heating and cooling demand is largely fixed by decisions made early in a building's design, yet the physical simulation used to estimate that demand is slow and ill-suited to rapid iteration. A machine-learning surrogate can return the same estimate in milliseconds, but a model on its own is not a usable system. This project, BEAM (Building Energy Assessment Model), treats the surrogate as one component of a complete, reproducible pipeline rather than as the deliverable. We train and compare seven regressors on the UCI Energy Efficiency dataset, which describes 768 simulated building shapes through eight design parameters and two load targets. Gradient-boosted trees dominate the linear baselines; the deployed XGBoost model reaches a validation R^2 of 0.9952 and a root-mean-squared error of 0.672 across both targets. Around this model we build seven containerised services launched with a single command: a FastAPI inference service, a PostgreSQL metrics store, MLflow for run tracking, Prefect for orchestration, Adminer and Grafana for inspection, and a training stage. A monitoring subsystem measures input distribution shift with a Kolmogorov–Smirnov statistic averaged over the eight features and raises an alert above a tuned threshold of 0.20; on a 32-batch run it classified 30 batches correctly with no false alarms. Reproducibility rests on two artifacts: versioned source on GitHub and versioned, runnable container images on the GitHub Container Registry. We also describe a continuous-integration pipeline whose tests deliberately target the seams between services.**

***Index Terms*—MLOps, building energy assessment, regression, XGBoost, data drift, Kolmogorov–Smirnov test, FastAPI, Docker, continuous integration**

I. Introduction

Buildings account for a substantial fraction of global energy consumption, and a large part of that demand is determined before construction begins, by choices of geometry, envelope, and glazing. Designers who want to compare options need fast feedback on how each variant will perform. The conventional route is a building-physics simulation, which is accurate but too slow to sit inside a tight design loop. A data-driven surrogate trained on simulation output offers an attractive alternative: once fitted, it predicts heating and cooling load almost instantly, so a designer can sweep through dozens of configurations in the time a single simulation would take.

The predictive task itself is well defined. Given eight numerical descriptors of a building, estimate two continuous quantities, the heating load and the cooling load. On the UCI Energy Efficiency benchmark this is a small, clean regression problem, and strong accuracy is achievable with standard methods. That is precisely why the model is not where the difficulty lies. A fitted estimator sitting in a notebook cannot be queried by other software, leaves no record of how it was produced, degrades silently when the inputs it receives drift away from what it was trained on, and cannot be reproduced by anyone who was not at the keyboard when it was trained. These are the problems that consume real effort in deployed machine learning, and they are largely orthogonal to model accuracy [7].

BEAM is built around that observation. The regression model is treated as one replaceable part of a larger system whose job is to train it reproducibly, serve it over HTTP, record every prediction, watch for distribution shift, and rebuild itself from version control on demand. The contributions of this work are the following. First, we package the full pipeline as seven containerised services that a single `docker-compose` command brings up in the correct order. Second, we compare seven regressors under a common protocol with MLflow tracking and deploy the strongest, an XGBoost model with a validation R^2 of 0.9952. Third, we add a monitoring subsystem that quantifies input drift with a Kolmogorov–Smirnov statistic and surfaces it on a live Grafana dashboard, achieving 93.8% drift-detection accuracy with zero false positives on a representative run. Fourth, we describe a continuous-integration and delivery pipeline that tests the integration boundaries between services and publishes versioned images to the GitHub Container Registry (GHCR), so that reproducibility derives from versioned code together with versioned, runnable images rather than from any single developer's machine.

II. Background and Related Work

The use of machine learning as a surrogate for building-energy simulation was established by Tsanas and Xifara, who generated the dataset used here from a parametric set of building shapes and showed that statistical models recover the simulated loads accurately [1]. Their work frames the task as supervised regression from a compact set of design parameters and remains the standard reference for this benchmark.

Among regression methods, gradient-boosted decision trees have proven particularly effective on tabular data of this kind. XGBoost [2] is a regularised, scalable implementation of gradient boosting that repeatedly fits trees to the residuals of the current ensemble; it tends to capture the smooth but

nonlinear interactions present in physical data while remaining inexpensive to train. The classical baselines we compare against, together with the boosting and forest models, are all provided by scikit-learn [3], which also supplies the evaluation metrics and the train/validation partition.

The systems side of the project draws on two strands of prior work. The first is the recognition that the model is a small part of a production machine-learning system, and that the surrounding infrastructure, configuration, and data dependencies are where most maintenance cost accumulates [7]. The second is the literature on monitoring deployed models for distribution shift. Concept and data drift, and the strategies for detecting and adapting to them, are surveyed by Gama et al. [5]. For detecting a change in a single feature's distribution, the two-sample Kolmogorov–Smirnov test is a long-established nonparametric tool: it measures the largest gap between two empirical cumulative distribution functions and is bounded between zero and one [4]. We use MLflow [6] to record training runs and their metrics, and Docker [8] to containerise every component so that the same image runs identically in development, in continuous integration, and on any host.

III. Problem Statement and Requirements

The core predictive problem is a two-output regression: map a vector of eight building-design parameters to the heating load and the cooling load. Around that problem, the project sets a number of system requirements that shaped every later decision.

The pipeline must be *reproducible*: a third party should be able to obtain the code, build the system, and arrive at the same trained model and the same running services without manual intervention. It must be *serveable*: predictions must be available to other software through a documented HTTP interface, not only inside a notebook. It must be *observable*: every prediction and its error must be persisted, and the model's behaviour over time must be visible on a dashboard. It must be *drift-aware*: the system must quantify how far the inputs it receives have moved from the training distribution and flag batches that move too far. And it must be *automatically verified*: a change pushed to version control must trigger tests that catch the kinds of breakage a multi-service system is prone to, and a verified change must produce a runnable, versioned artifact.

These requirements are deliberately weighted toward the system rather than the model. The dataset is small and the regression is tractable, so the accuracy requirement, while real, is the least demanding of the set.

IV. Data

The experiments use the UCI Energy Efficiency dataset [1], distributed as an Excel workbook of 768 rows. Each row corresponds to one simulated residential building and contains eight input features and two output targets, summarised in Table I. The features describe the building's shape and envelope: its relative compactness, surface area, wall area, roof area, overall height, orientation, glazing area, and the distribution of that glazing. The two targets, the heating load and the cooling load, are the quantities the model must predict.

TABLE I
Features and targets of the Energy Efficiency dataset.

Symbol	Meaning	Role
X1	Relative compactness	feature
X2	Surface area	feature
X3	Wall area	feature
X4	Roof area	feature
X5	Overall height	feature
X6	Orientation	feature
X7	Glazing area	feature
X8	Glazing area distribution	feature
Y1	Heating load	target
Y2	Cooling load	target

Preprocessing is intentionally light, because the data is already clean and numerical. Rows with missing values are dropped as a safeguard, and the eight features and two targets are separated. The data is then partitioned into 80% for training and 20% for validation with a fixed random seed, giving roughly 614 training rows and 154 validation rows. The seed makes the partition identical on every run, which is what allows the seven models to be compared on equal terms: any difference in their scores comes from the model, not from a different slice of data. No separate held-out test set is reserved; the implications of this are discussed in Section IX.

V. Methodology and Models

The modelling stage trains seven regressors on the training partition and scores each on the validation partition. The candidates span three families: linear models (ordinary least squares, Ridge with $\alpha = 1.0$, and Lasso with $\alpha = 0.1$), a single decision tree, and ensembles (a random forest and a gradient-boosting regressor, each with 100 estimators, and an XGBoost regressor with 100 estimators). Because the task has two outputs, the gradient-boosting and XGBoost regressors, which are natively single-output, are wrapped so that one estimator is fitted per target; the linear models, the tree, and the forest handle both outputs directly. All models use a fixed random seed for reproducibility.

Each model is evaluated with four metrics: the coefficient of determination R^2 and the root-mean-squared error (RMSE) computed jointly over both targets, and the per-target R^2 for heating and for cooling separately. The per-target figures matter because a model can be strong on one load and weak on the other, and the joint score alone would hide that.

Every run is logged to MLflow [6], which records the model name, the four metrics, and the fitted model artifact under a single named experiment. Figure 1 shows the resulting tracking view. After all seven runs complete, the pipeline selects the model with the highest validation R^2, serialises it to disk, and that artifact becomes the one the inference service loads. On every run observed in this project the selected model was XGBoost.

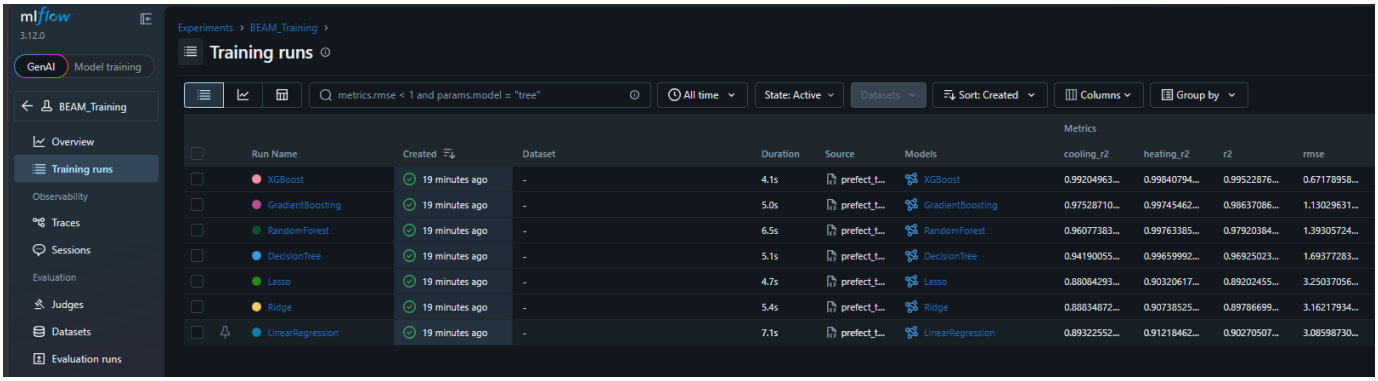


Fig. 1. MLflow tracking view of the `BEAM_Training` experiment. Each of the seven regressors is logged as a separate run with its validation metrics (`cooling_r2`, `heating_r2`, `r2`, `rmse`) and training duration. XGBoost records the highest R^2 (0.9952) and the lowest RMSE (0.672); the gradient-boosting and forest models follow, and the linear models trail. The runs were produced by the Prefect training flow.

VI. System Architecture and Infrastructure

BEAM is composed of seven services defined in a single Compose file and started together with `docker-compose up --build`. Four of the seven, the training stage, the inference API, the MLflow server, and the monitoring stage, are built from one shared Docker image [8] and differ only in the command they run; the remaining three, PostgreSQL, Adminer, and Grafana, use stock images. Reusing a single image for the project's own code keeps the dependency set consistent across roles and means a rebuild touches one definition rather than four.

Data flows through the system in one direction. The training stage reads the dataset, fits the seven models, and writes the best one to a shared volume. The inference service loads that model once at start-up and holds it in memory together with the training data, which it keeps as the reference distribution for drift measurement. Incoming requests are scored against the model; batch requests additionally have their metrics and per-row predictions written to PostgreSQL. Grafana reads from the same database to render its panels. Service start-up order is enforced by health checks so that, for example, the API waits for both the training stage to finish and the database to become ready before it accepts traffic.

A. Model serving with FastAPI

The model is served by a FastAPI application that exposes three endpoints, shown in the generated OpenAPI documentation in Figure 2. A root endpoint reports service health; `/predict` scores a single building; and `/log_batch` scores a batch, computes metrics and drift, and persists the result. Request and response shapes are declared as typed schemas (`BuildingFeatures`, `BatchRow`, `BatchPayload`), so malformed input is rejected automatically with a validation error rather than reaching the model.

Figure 3 shows a single prediction. The eight features of one building are posted to `/predict`, and the service returns a heating load of 15.56 and a cooling load of 21.33; these values essentially reproduce the known targets for that building,

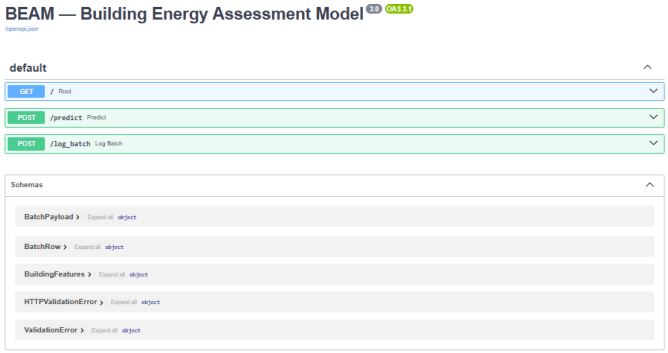


Fig. 2. FastAPI Swagger UI for the BEAM service. Three endpoints are exposed: `GET /` (health), `POST /predict` (single prediction), and `POST /log_batch` (batch scoring with metric and drift logging). The typed request and response schemas are listed below the endpoints.

which is the expected behaviour for an accurate model on an in-distribution input.

The batch endpoint, shown in Figure 4, returns a richer response: a batch identifier, the sample count, the regression metrics, the measured drift score, and a Boolean alert flag with the threshold against which it was decided. The figure illustrates the response contract on a deliberately minimal three-row payload; the metrics are not statistically meaningful at that size, which is itself a reminder that the production monitoring batches use fifty samples.

B. Persistence with PostgreSQL

PostgreSQL stores two tables, populated by the monitoring stage and by the batch endpoint. The schema is applied automatically when the database container first starts. The `model_metrics` table holds one row per batch, shown in Figure 5: the batch identifier and timestamp, the model name, the sample count, the joint R^2, RMSE and MAE, the per-target R^2 values, the injected drift level, and the measured drift score. The `prediction_log` table holds one row per individual prediction, shown in Figure 6: the actual and predicted heating and cooling values and their errors. At the time of capture this

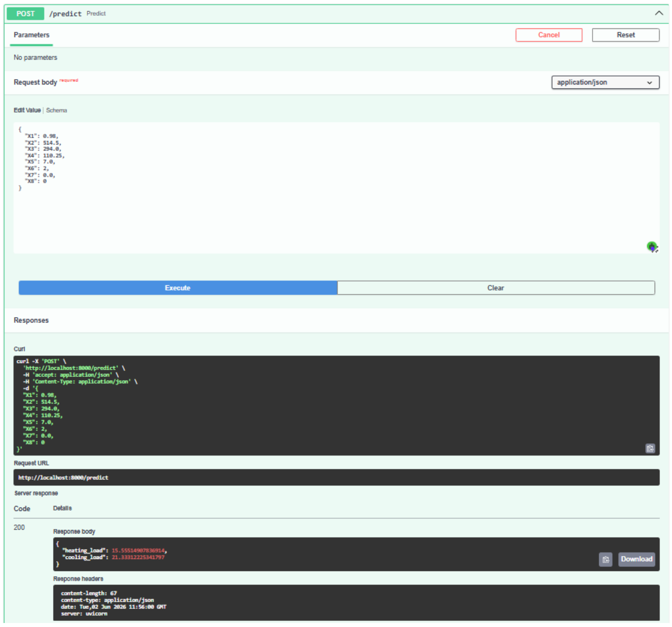


Fig. 3. A /predict call. The request body carries the eight design parameters of one building; the response returns the predicted heating load (15.56) and cooling load (21.33). The server responds with HTTP 200.

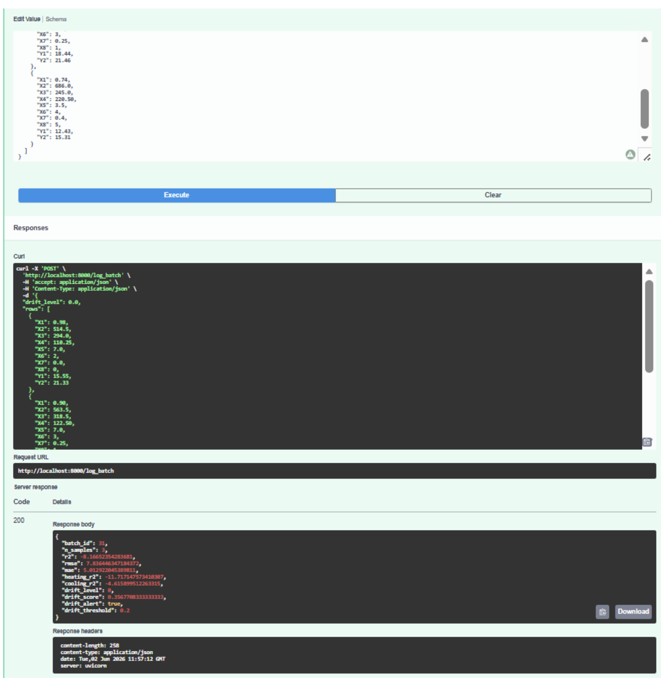
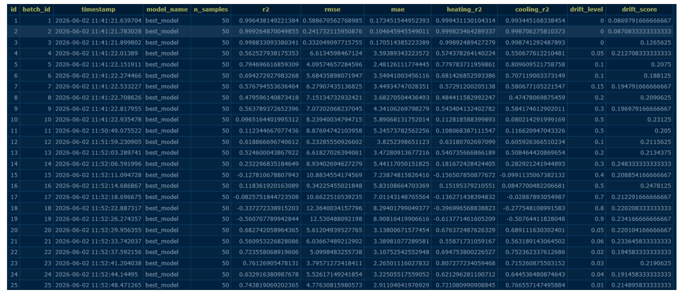


Fig. 4. A /log_batch call and its response contract. The service returns the batch identifier, sample count, regression metrics, the averaged Kolmogorov–Smirnov drift score, and the alert flag with its 0.20 threshold. The three-row payload shown is a minimal contract check; metric values are unstable at this sample size.

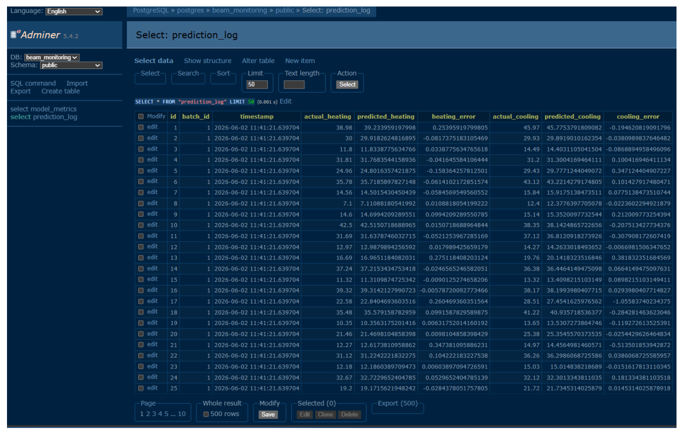
table held 500 rows, one for every individual prediction across the logged batches.

Adminer is attached to the same database purely as a lightweight inspection tool, which made it straightforward to verify during development that the application's writes matched the declared schema.



id	batch_id	timestamp	model_name	n_samples	r2	rmse	mae	heating_r2	cooling_r2	drift_level	drift_score
1	1	2026-06-02 11:41:21.639704	best_model	50	0.996438149221384	0.588670562768985	0.173451544952393	0.999431130104314	0.993445168338454	0	0.0869791666666667
2	2	2026-06-02 11:41:21.783028	best_model	50	0.999264870049855	0.241732115950876	0.104645945549011	0.999823464289337	0.998706275810373	0	0.0870833333333333
3	3	2026-06-02 11:41:21.899802	best_model	50	0.998833093380341	0.332049097715755	0.170514385223389	0.99892489427279	0.998741292487893	0	0.1265625
4	4	2026-06-02 11:41:22.01389	best_model	50	0.562527938175353	6.6134598467124	3.59389343223572	0.574378264140224	0.550677612210481	0.05	0.212708333333333
5	5	2026-06-02 11:41:22.151911	best_model	50	0.794696616859309	4.09574657284596	2.48126111774445	0.779783711959861	0.809609521758758	0.1	0.2075
6	6	2026-06-02 11:41:22.274466	best_model	50	0.694272927983268	5.68435898071947	3.54941003456116	0.681426852593386	0.707119003373149	0.1	0.188125
7	7	2026-06-02 11:41:22.533227	best_model	50	0.576794553636464	6.27907435136825	3.44934747028351	0.57291200205138	0.580677105221547	0.15	0.194791666666667
8	8	2026-06-02 11:41:22.708626	best_model	50	0.479596140873418	7.15134732932421	3.68270504436493	0.484411582992247	0.47478069875459	0.2	0.2090625
9	9	2026-06-02 11:41:22.817955	best_model	50	0.563789372652396	7.07202068237045	4.34106269798279	0.543404132402782	0.584174612902011	0.3	0.196979166666667
10	10	2026-06-02 11:41:22.935478	best_model	50	0.0965164401995312	8.23940034794715	5.89068131752014	0.112818588399893	0.080214291999169	0.5	0.23125
11	11	2026-06-02 11:50:49.075522	best_model	50	0.112344667077436	8.87694742103958	5.24573782562256	0.108068387111547	0.116620947043326	0.5	0.205
12	12	2026-06-02 11:51:59.230905	best_model	50	0.618866696740612	6.23285550926602	3.8252398651123	0.63180702697099	0.605926366510234	0.1	0.2115625
13	13	2026-06-02 11:52:03.289741	best_model	50	0.524600043867922	6.61827026394061	3.47280913677216	0.540735666866189	0.508464420869654	0.2	0.2134375
14	14	2026-06-02 11:52:06.591996	best_model	50	0.232296835184649	8.93402694627279	5.44117050151825	0.181672428424405	0.282921241944893	0.3	0.248333333333333
15	15	2026-06-02 11:52:11.094728	best_model	50	-0.127810678807943	10.8834554174569	7.23874815826416	-0.156507850877672	-0.0991135067382132	0.4	0.208854166666667
16	16	2026-06-02 11:52:14.686867	best_model	50	0.118361920163089	9.34225455021848	5.83108664703369	0.15195379210551	0.0847700482206681	0.5	0.2478125
17	17	2026-06-02 11:52:18.696675	best_model	50	-0.0825751844723508	10.6622510539235	7.01143148765564	-0.136271438394832	-0.02887893054987	0.7	0.212291666666667
18	18	2026-06-02 11:52:22.887317	best_model	50	-0.337272338915203	12.3640034157796	8.29401799049377	-0.396996568838823	-0.277548108991583	0.8	0.220208333333333
19	19	2026-06-02 11:52:26.274357	best_model	50	-0.560707789942844	12.530488092198	8.90816419906616	-0.613771461605209	-0.50764411828048	0.9	0.234166666666667
20	20	2026-06-02 11:52:29.956355	best_model	50	0.682742058964365	5.61204939527765	3.13800671577454	0.676372487626329	0.689111630302401	0.05	0.220104166666667
21	21	2026-06-02 11:52:33.742037	best_model	50	0.560953226828086	6.03667489212902	3.38981077289581	0.55871731059167	0.563189143064502	0.06	0.233645833333333
22	22	2026-06-02 11:52:37.592156	best_model	50	0.723558068919606	5.0998483255738	3.10752542552948	0.694753800226527	0.752362337612686	0.02	0.194583333333333
23	23	2026-06-02 11:52:41.204038	best_model	50	0.76126905478131	3.79571272418411	2.26501116027832	0.807277234059468	0.715260875503152	0.03	0.2190625
24	24	2026-06-02 11:52:44.14495	best_model	50	0.632916380987678	5.52617149241854	3.22505517559052	0.621296281100712	0.644536480874643	0.04	0.191458333333333
25	25	2026-06-02 11:52:48.471265	best_model	50	0.743819069202365	4.77630815980573	2.91104041976929	0.721080990908845	0.766557147495884	0.01	0.214895833333333

Fig. 5. The model_metrics table in Adminer. Each row is one batch. The early batches (drift level 0) show R^2 near 0.99 and a drift score below 0.13; once a nonzero drift level is injected (from batch 4), R^2 falls sharply and the drift score rises past the 0.20 threshold.



Modify	id	batch_id	timestamp	actual_heating	predicted_heating	heating_error	actual_cooling	predicted_cooling	cooling_error
edit	1	1	2026-06-02 11:41:21.639704	38.98	39.233959197998	0.25395919799805	45.97	45.7753791809082	-0.194620819091796
edit	2	1	2026-06-02 11:41:21.639704	30	29.9182624816895	-0.0817375183105469	29.93	29.8919010162354	-0.0380989837646482
edit	3	1	2026-06-02 11:41:21.639704	11.8	11.8338775634766	0.0338775634765618	14.49	14.4031105041504	-0.0868894958496096
edit	4	1	2026-06-02 11:41:21.639704	31.81	31.7683544158936	-0.041645584106444	31.2	31.3004169464111	0.100416946411134
edit	5	1	2026-06-02 11:41:21.639704	24.96	24.8016357421875	-0.158364257812501	29.43	29.7771244049072	0.347124404907227
edit	6	1	2026-06-02 11:41:21.639704	35.78	35.7185897827148	-0.061410217285l574	43.12	43.2214279174805	0.10142917480471
edit	7	1	2026-06-02 11:41:21.639704	14.56	14.5015430450439	-0.0584569549560552	15.84	15.9175138473511	0.0775138473510744
edit	8	1	2026-06-02 11:41:21.639704	7.1	7.11088180541992	0.0108818054199222	12.4	12.3776397705078	-0.0223602294921879
edit	9	1	2026-06-02 11:41:21.639704	14.6	14.6994209289551	0.0994209289550785	15.14	15.3520097732544	0.212009773254394
edit	10	1	2026-06-02 11:41:21.639704	42.5	42.5150718688965	0.0150718688964844	38.35	38.1424865722656	-0.207513427734376
edit	11	1	2026-06-02 11:41:21.639704	31.69	31.6378746032715	-0.0521253967285169	37.12	36.8120918273926	-0.307908172607419
edit	12	1	2026-06-02 11:41:21.639704	12.97	12.9879894256592	0.017989425659179	14.27	14.2633018493652	-0.0066981506347652
edit	13	1	2026-06-02 11:41:21.639704	16.69	16.9651184082031	0.275118408203124	19.76	20.1418323516846	0.381832351684569
edit	14	1	2026-06-02 11:41:21.639704	37.24	37.2153434753418	-0.0246565246582051	36.38	36.4464149475098	0.0664149475097631
edit	15	1	2026-06-02 11:41:21.639704	11.32	11.3109874725342	-0.0090125274658206	13.32	13.4098215103149	0.0898215103149411
edit	16	1	2026-06-02 11:41:21.639704	39.32	39.3142127990723	-0.00578720092773466	38.17	38.1993980407715	0.0293980407714827
edit	17	1	2026-06-02 11:41:21.639704	22.58	22.8404693603516	0.260469360351564	28.51	27.4541625976562	-1.05583740234375
edit	18	1	2026-06-02 11:41:21.639704	35.48	35.579158782959	0.0991587829589875	41.22	40.935718536377	-0.284281463623046
edit	19	1	2026-06-02 11:41:21.639704	10.35	10.3563175201416	0.00631752014160192	13.65	13.5307273864746	-0.119272613525391
edit	20	1	2026-06-02 11:41:21.639704	21.46	21.4698104858398	0.0098104858398429	25.38	25.3545570373535	-0.0254429626464834
edit	21	1	2026-06-02 11:41:21.639704	12.27	12.6173810958862	0.347381095886231	14.97	14.4564981460571	-0.513501853942872
edit	22	1	2026-06-02 11:41:21.639704	31.12	31.2242221832275	0.104222183227538	36.26	36.2986068725586	0.0386068725585957
edit	23	1	2026-06-02 11:41:21.639704	12.18	12.1860389709473	0.00603897094726591	15.03	15.014838218689	-0.0151617813110345
edit	24	1	2026-06-02 11:41:21.639704	32.67	32.7229652404785	0.0529652404785139	32.12	32.3013343811035	0.181334381103518
edit	25	1	2026-06-02 11:41:21.639704	19.2	19.1715621948242	-0.0284378051757805	21.72	21.7345314025879	0.0145314025878918

Fig. 6. The prediction_log table in Adminer, holding 500 rows. Each row records one building's actual and predicted heating and cooling loads and the signed errors, which feed the prediction-error panel of the dashboard.

C. Drift detection

For each incoming batch the service compares every feature's distribution against the corresponding feature in the reference (training) data using the two-sample Kolmogorov–Smirnov statistic [4]. The eight per-feature statistics are averaged into a single drift score, bounded in $[0, 1]$, and a score at or above 0.20 raises an alert. Scaling drift by each feature's own variability and reducing it to one number keeps the signal interpretable on a dashboard while remaining sensitive to shifts in any single input.

D. Orchestration, containerisation, and CI/CD

The training and monitoring stages are written as Prefect flows, which gives each step structured logging and clear task boundaries at the cost of a couple of decorators; Prefect runs in an ephemeral mode that needs no persistent server. Every component is containerised with Docker [8].

Continuous integration runs on every push through GitHub Actions in three parallel jobs. A test job runs the linter and a suite of unit and integration tests; a schema job starts a real PostgreSQL service container, applies the schema, and

executes the exact insert statements the application uses, so that any divergence between the schema and the code fails the build; and a Docker job builds the image and confirms that every module imports inside it. The tests are written to target integration boundaries rather than isolated functions: that the feature list is identical across the four modules that declare it, that the database schema matches the code's inserts, that the drift statistic behaves correctly and stays bounded on edge cases, and that the batch payload the simulator produces matches the schema the API expects. Twenty-seven tests run locally, with three further schema tests activated against the Postgres service in CI. On a successful push to the main branch, a delivery workflow builds the image and publishes it to GHCR, tagged with `latest`, the commit hash, and the branch name. Reproducibility therefore comes from two versioned artifacts in combination: the source on GitHub and the runnable image on GHCR.

VII. Results

A. Model performance

Table II reports the validation metrics for all seven models, ordered by R^2. The separation between model families is pronounced. The three tree ensembles and the single tree all exceed an R^2 of 0.96, whereas the three linear models sit near 0.90. XGBoost is the strongest on every metric that matters for deployment, with a joint R^2 of 0.9952 and an RMSE of 0.672, and it is also the fastest of the ensembles to train at 4.1 seconds.

TABLE II
VALIDATION PERFORMANCE OF ALL SEVEN MODELS, ORDERED BY R^2. HEATING AND COOLING COLUMNS ARE PER-TARGET R^2. TRAINING TIME IS FROM THE MLFLOW RUNS.

Model	R^2	RMSE	Heating	Cooling	Time
XGBoost	**0.9952**	**0.672**	0.9984	0.9920	4.1 s
Gradient Boosting	0.9864	1.130	0.9975	0.9753	5.0 s
Random Forest	0.9792	1.393	0.9976	0.9608	6.5 s
Decision Tree	0.9693	1.694	0.9966	0.9419	5.1 s
Linear Regression	0.9027	3.086	0.9122	0.8932	7.1 s
Ridge	0.8979	3.162	0.9074	0.8883	5.4 s
Lasso	0.8920	3.250	0.9032	0.8808	4.7 s

The same comparison is shown graphically in Figure 7. A consistent pattern runs through all four panels: every model predicts the heating load more accurately than the cooling load, and the gap is widest for the weaker models. Cooling load is the harder target here, and it is where the linear models lose the most ground. The RMSE panel makes the practical difference vivid: XGBoost's error of 0.672 is roughly a fifth of the linear models' error of just over three.

B. Monitoring and drift detection

The operational behaviour of the system is captured by the Grafana dashboard in Figure 8, which refreshes every ten seconds and reads directly from PostgreSQL. Its seven panels track R^2, RMSE, the measured drift score against the injected drift level, the per-target R^2 split, drift level against RMSE, a per-batch metrics table, and the error distribution of the most recent batch.

The dashboard demonstrates the central claim of the monitoring subsystem: the measured drift score tracks the injected drift level, and model accuracy falls as drift rises. On clean batches the model holds an R^2 near 0.99 with a drift score below 0.13; once drift is injected, the score climbs past the 0.20 line and R^2 degrades, in places turning negative. The relationship between the injected level and the measured score is what validates the detector: it is responding to genuine distribution shift rather than to noise.

Treating a batch as drifted when its injected level is nonzero, and as detected when its score reaches 0.20, we evaluated the detector across a 32-batch run spanning injected levels from 0 to 1.0. It classified 30 of the 32 batches correctly, an accuracy of 93.8%, with 24 true positives, 6 true negatives, no false positives, and 2 false negatives. The absence of false positives is the result that matters most for an alerting system: on this run, every batch the detector flagged was genuinely drifted, so the alert can be trusted. The two missed batches sat at the threshold boundary, which reflects a deliberate bias toward precision over recall.

VIII. Discussion

Two things stand out from the results. The first is how decisively the tree ensembles beat the linear models on this data. The relationship between a building's geometry and its energy load is smooth but not linear, and the boosting models capture the curvature that the linear models cannot; XGBoost's fifth-of-the-error advantage on RMSE is the clearest expression of that. The per-target panels add a useful nuance, namely that cooling load is consistently the harder of the two outputs, which is where the weaker models fall furthest behind and where the per-target metric earns its place.

The second is that the monitoring subsystem behaves as designed, and the design choices behind it are defensible. Averaging the per-feature Kolmogorov–Smirnov statistics into one score trades some diagnostic detail for a number a viewer can read at a glance, and pairing it with the injected drift level on the same panel turns the dashboard into direct evidence that the detector tracks real shift. Setting the alert threshold at 0.20 reflects a deliberate preference: on the evaluation run it produced no false alarms, at the cost of missing two borderline batches. For an alert that a team is meant to act on, that trade is the right way round, because an alarm that fires only when something is genuinely wrong is one people keep trusting. A related point concerns the dashboard itself: a monitoring view is only useful if it can be read at a glance, so the panels were kept to the few quantities that actually drive a decision, namely overall and per-target accuracy, error magnitude, and the measured drift against the injected level, rather than every column the database stores. Adding panels without restraint would bury the one signal the dashboard exists to surface.

The broader point is the one the project set out to make. The accuracy in Table II took a small fraction of the effort that went

Fig. 7. Per-model validation performance. Top-left: RMSE (lower is better), where XGBoost reaches 0.67 against roughly 3.1–3.3 for the linear models. Top-right: joint R^2. Bottom-left: heating-load R^2 (Y1), on which all tree models are essentially saturated. Bottom-right: cooling-load R^2 (Y2), the harder target, where the spread between model families is widest.

into the system around it: the service contract, the schema, the orchestration, the dashboard, and the tests that hold the pieces together. That distribution of effort is the realistic shape of deployed machine learning, and building the system end to end made the integration boundaries between components visible in a way that training a model in isolation never would.

IX. Limitations and Future Work

Several limitations are worth stating plainly. The monitoring batches are sampled from the same reference distribution the model was trained on, with synthetic noise injected to simulate drift. This is appropriate for demonstrating and validating the detection mechanism under controlled conditions, but it is not the same as monitoring a stream of genuinely unseen production data; a natural next step is to feed the monitor from a rolling window of live inference traffic and to maintain a held-out reference rather than reusing the training set. A fixed sliding window over the most recent requests, for example the last one thousand, or an exponentially weighted moving average would let that reference track recent traffic without growing without bound. The 93.8% detection accuracy should accordingly be read as the detector's behaviour under controlled, synthetically induced shift, and would need to be re-established against genuine traffic before any operational claim is made on it.

The model is evaluated on a single 80/20 split rather than with cross-validation, and no separate test set is held back beyond the validation partition used for model selection. On a dataset of 768 rows this keeps each partition usable, but a k-fold cross-validation protocol, for which five or ten folds would be appropriate at this size, combined with a genuinely untouched test set, would give a less optimistic and more stable estimate of generalisation. Because the same validation partition served both to select the best model and to report its score, the figures in Table II are best read as validation performance that may be mildly optimistic rather than as an unbiased estimate on unseen data. The drift detector uses one static threshold applied to the mean of the per-feature statistics; per-feature thresholds, or a threshold recalibrated against a baseline window, would be more sensitive to drift concentrated in a single input. Finally, we report training durations but did not formally measure inference latency or throughput under load, which would be needed to characterise the service operationally; we leave that measurement as future work.

X. Conclusion

BEAM approaches building energy load prediction as a systems problem rather than a modelling one. The regression itself is solved comfortably: an XGBoost model reaches a validation R^2 of 0.9952 and an RMSE of 0.672, clearly ahead of the linear baselines. The contribution of the project is the reproducible pipeline built around that model, seven containerised services that train it, serve it over a typed HTTP interface, persist every prediction, and watch the input distribution for drift, all launched with one command and all rebuilt automatically from version control. The monitoring subsystem detects injected drift with 93.8% accuracy and no false positives, and the continuous-integration pipeline guards the seams between services where a multi-component system

Fig. 8. The BEAM monitoring dashboard. The clean early batches hold R^2 near 0.99 with a drift score around 0.09. As injected drift increases, the measured drift score (green) tracks the injected level (yellow) and crosses the 0.20 threshold, while R^2 collapses and RMSE climbs. The batch-metrics table records, for example, batch 1 at R^2=0.996 with drift score 0.087, and batch 4 at R^2=0.563 once a drift level of 0.05 is introduced.

is most likely to break. Reproducibility rests on versioned source on GitHub together with versioned, runnable images on GHCR. The result is a model that is not merely accurate in a notebook but deployable, observable, and reproducible as a running system.

XI. Code and Artifact Availability

All code, configuration, and documentation are available in a public GitHub repository, shared with the lecturer for evaluation:

https://github.com/ProNilabh/beam-project

The repository structure and commit history are shown in Figure 9. Versioned, runnable container images are published to the GitHub Container Registry:

https://github.com/ProNilabh/beam-project/pkgs/container/beam-project

The published image can be pulled directly, for example with `docker pull ghcr.io/pronilabh/beam-project:latest`, and the full stack is brought up from the repository with `docker-compose up --build`. The dataset is the public UCI Energy Efficiency dataset [1] and is read from the repository's data directory at run time; it is not redistributed as a separate artifact.

XII. Generative AI Declaration

In line with the course policy, I declare that generative AI tools were used during this project. They assisted with writing and debugging code, with clarifying details of libraries such as FastAPI, Prefect, and SQLAlchemy, and with improving the wording and structure of this report. I did not use AI to generate results. The design of the pipeline, the choice of models and their hyper-parameters, the training and monitoring runs, the database schema, and the analysis of the outputs are my own work. Every number reported here was produced by my own code and recorded by MLflow, the PostgreSQL store, or the running service. I have reviewed and understood all of the code and the content of this report and can explain and defend each part of it.

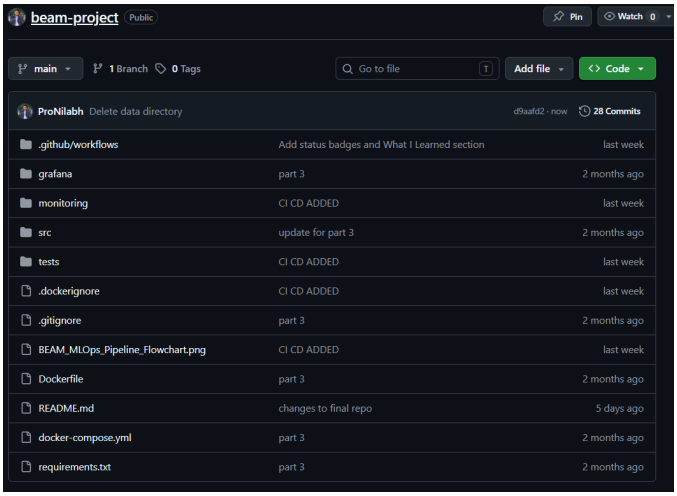


Fig. 9. The public GitHub repository for BEAM. The layout separates the workflow definitions (`.github/workflows`), the Grafana provisioning, the monitoring code, the source, and the tests, alongside the Dockerfile, Compose file, and requirements. The history shows 28 commits across the project's parts.

REFERENCES

[1] A. Tsanas and A. Xifara, "Accurate quantitative estimation of energy performance of residential buildings using statistical machine learning tools," *Energy and Buildings*, vol. 49, pp. 560–567, 2012.

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[4] F. J. Massey, "The Kolmogorov–Smirnov test for goodness of fit," *J. Amer. Statist. Assoc.*, vol. 46, no. 253, pp. 68–78, 1951.

[5] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.

[6] M. Zaharia et al., "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, 2018.

[7] D. Sculley et al., "Hidden technical debt in machine learning systems," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 2503–2511.

[8] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, vol. 2014, no. 239, 2014.